

Documentacion Tecnica - Proyecto Cordoba

1. Que hace el proyecto

Este proyecto es una aplicacion de escritorio hecha en **C++**. Se armo para representar algunas ciudades de Cordoba en un mapa y poder calcular la ruta mas corta entre dos puntos.

Para resolverlo use la idea de un **grafo**:

- ciudad = nodo;
- ruta = conexion;
- distancia = peso de la conexion;
- algoritmo usado = Dijkstra.

2. Archivos principales

Archivo	Para que sirve
ProyectoCordobaUI.cpp	Es el punto de inicio de la aplicacion.
Nodo.h	Tiene la estructura que uso para guardar una ciudad.
Grafo.h/.cpp	Maneja las ciudades, las conexiones, Dijkstra y el guardado.
DatosIniciales.cpp	Carga las ciudades y rutas con las que arranca el programa.
UI.h/.cpp/.dfm	Contiene la pantalla principal, el mapa y los botones principales.
FormNodos.h/.cpp	Contiene la ventana para administrar ciudades.
FormConexiones.h/.cpp	Contiene la ventana para administrar conexiones.
mapaCordoba3.jpg	Es la imagen que se usa como fondo del mapa.

3. Estructura de una ciudad

```
struct Nodo
{
    int id;
    char nombre[50];
    int x;
    int y;
    int xPX;
    int yPX;
```

```
bool activo;  
};
```

Los campos que mas se usan son:

- `id` : identifica la ciudad dentro del arreglo;
- `nombre` : es el texto que se ve en pantalla;
- `xPX` e `yPX` : indican donde se dibuja la ciudad en el mapa;
- `activo` : indica si la ciudad esta disponible o no.

4. Como se guarda el grafo

Para guardar las rutas use una matriz:

```
int matriz[NODOS_CANTIDAD][NODOS_CANTIDAD];
```

El proyecto trabaja con una cantidad fija de nodos:

```
#define NODOS_CANTIDAD 8
```

Cada valor de la matriz tiene un significado:

Valor	Significado
0	Es la misma ciudad.
> 0	Hay conexion activa.
-1	No hay conexion.
< -1	La ruta existe, pero esta cortada.

Ejemplo:

```
matriz[0][1] = 140;  
matriz[1][0] = 140;
```

Esto quiere decir que las ciudades 0 y 1 estan conectadas a 140 km.

5. Ciudades iniciales

ID	Ciudad	Estado inicial
0	Cordoba	Activa
1	Villa Maria	Activa
2	Rio Cuarto	Inactiva
3	Carlos Paz	Activa
4	San Francisco	Inactiva
5	Bell Ville	Activa
6	Alta Gracia	Activa
7	Rio Tercero	Activa

Rio Cuarto y San Francisco quedan inactivas al principio para poder probar el alta de ciudades desde el formulario.

6. ABM de ciudades

El ABM de ciudades esta hecho en `FormNodos` .

Permite:

- dar de alta ciudades inactivas;
- modificar nombre o coordenadas;
- dar de baja una ciudad activa.

La baja es logica. O sea, no borro el nodo del arreglo, sino que cambio su estado:

```
nodos[id].activo = false;
```

De esa forma la ciudad puede volver a activarse mas adelante.

7. ABM de conexiones

El ABM de conexiones esta hecho en `FormConexiones` .

Permite:

- agregar una ruta;
- modificar distancia;
- eliminar una ruta.

Para agregar o modificar se usa:

```
conectar(origen, destino, distancia);
```

Para eliminar:

```
eliminarConexion(origen, destino);
```

Cuando elimino una conexion, la posicion de la matriz queda en `-1`, o sea, sin ruta directa.

8. Cortar y restaurar rutas

Para cortar o restaurar una ruta uso:

```
cortarORestaurarConexion(origen, destino);
```

La logica que use es esta:

```
140  -> -140  ruta cortada  
-140 -> 140   ruta restaurada
```

Con esto no pierdo la distancia original: solamente cambio el signo para saber si la ruta esta cortada o activa.

9. Dijkstra

Dijkstra es el algoritmo que uso para buscar la ruta mas corta entre el origen y el destino.

Para hacerlo uso tres arreglos:

```
int  distancias[NODOS_CANTIDAD];  
bool visitado[NODOS_CANTIDAD];  
int  anterior[NODOS_CANTIDAD];
```

Una de las condiciones mas importantes es:

```
nodos[i].activo && matriz[nodoActual][i] > 0
```

Eso hace que Dijkstra solo tenga en cuenta:

- ciudades activas;
- conexiones activas;
- distancias positivas.

Por eso, si una ruta esta cortada y queda con distancia negativa, el algoritmo directamente la ignora.

10. Pantalla principal

Desde la pantalla principal se puede:

- elegir origen;
- elegir destino;
- buscar ruta;
- cortar o restaurar una ruta;
- abrir ABM de ciudades;
- abrir ABM de conexiones.

11. Guardado de datos

El programa guarda los datos en estos archivos:

Archivo	Guarda
nodos.dat	Ciudades y estados.
matriz.dat	Conexiones y distancias.
Historial.txt	Busquedas realizadas.